

MySQL, PostgreSQL, Derby and HSQLDB are the popular open source databases that are officially supported, and Oracle, MS-SQL, IBM DB2, and IBM Informix are among the proprietary databases supported.

Getting started

Before anything else, you should have your initial and expected datasets ready. DbUnit accepts data in the form of flat XML files, so if your table looks something like that in Figure 2, the resulting XML should be as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<dataset>
  <employee id="325" name="Rahul Chand" salary="10000"/>
  <employee id="326" name="Abhishek Kumar" salary="5000"/>
  <employee id="327" name="Pankaj Kumar" salary="7000"/>
  <employee id="329" name="Rohit Garg" salary="8000"/>
</dataset>
```

Here, each different child element within the `<dataset>` element represents the name of the table ('employee', in this case), and its attributes contain the data to be stored in each column of that table's row. You could manually write such a dataset if it is small—but for large datasets, you can extract a snapshot of data from a live/test database into flat XML files using DbUnit itself, with code like what's shown in the following block:

```
QueryDataSet partDS = new QueryDataSet(conn);
partDS.addTable("employee", "SELECT * FROM employee WHERE salary=5000");
FlatXmlDataSet.write(partDS, new FileOutputStream("employee-exported.xml"));
```

Here, the `conn` parameter passed to the `QueryDataSet` is an `IdatabaseConnection` object, which obtains a connection to the database. This `QueryDataSet` technique is very powerful in the sense that it can be made to create arbitrary datasets from the result of any SQL query, and you are bound to use it while writing your tests.

One thing to keep in mind while using DbUnit is that it does not automatically create the database schema and tables for you; it can only load data into existing tables, so you will need to have your database schema ready before trying to use DbUnit.

Through the code

To get started writing a test case, you can extend the `DBTestCase` class, which means you must also override the `getDataSet()` method, which should return the initial flat XML dataset to be loaded into the database.

```
protected IDataset getDataSet() throws Exception {
    return new FlatXmlDataSetBuilder().build(new
        FileInputStream("employee-init.xml"));
}
```

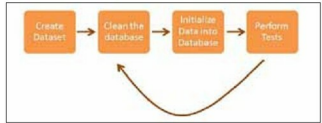


Figure 1: The DbUnit Workflow

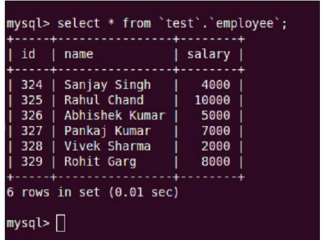


Figure 2: The sample MySQL Data

By default, DbUnit uses the `DriverManager`, which locates its configuration from within the system properties to obtain a JDBC connection to the database. This can be done in the constructor of the test-case class.

```
public hellotest(String name) {
    super(name);
    System.setProperty(PropertiesBasedJdbcDatabaseTester.DBUNIT_
        DRIVER_CLASS, "com.mysql.jdbc.Driver");
    System.setProperty(PropertiesBasedJdbcDatabaseTester.DBUNIT_
        CONNECTION_URL, "jdbc:mysql://localhost/test"); System.setProperty
        (PropertiesBasedJdbcDatabaseTester.DBUNIT_USERNAME, "root");
    System.setProperty(PropertiesBasedJdbcDatabaseTester.DBUNIT_
        PASSWORD, "vampire");
    System.setProperty(PropertiesBasedJdbcDatabaseTester.DBUNIT_
        SCHEMA, "test");
}
```

Now, you are ready to implement whatever test methods you need, and perform assertions using methods provided by DbUnit. One sample implementation of such a method, which tests the insertion of a record in the aforementioned database, is as follows:

```
Employee emp = new Employee();
emp.insert();
IDataset expds = new FlatXmlDataSetBuilder().build(new
    FileInputStream("employee-expect.xml"));
```